

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: B. Charishma

Roll No.: A24126552069

Date: 31-08-2026

1. TITLE

Design and Implementation of an Interactive To-Do List Application Using JavaScript DOM Manipulation and Event Handling

2. AIM

To design and develop a dynamic To-Do List web application using HTML5, CSS3, and JavaScript that allows users to create new tasks, mark tasks as completed via dynamic class toggling, delete tasks from the Document Object Model (DOM), and maintain responsive empty list state indicators.

3. OBJECTIVE

The objectives of this experiment are:

- To access and manipulate Document Object Model (DOM) elements using JavaScript (`document.getElementById()`, `document.createElement()`, `appendChild()`, `remove()`).
 - To implement event-driven programming using `addEventListener("click", ...)`.
 - To perform client-side input validation and whitespace trimming (`trim()`).
 - To dynamically toggle CSS styles using `classList.toggle("completed")` for task status representation.
 - To inspect DOM child nodes (`taskList.children.length`) and toggle empty list placeholder messages.
 - To clear user input fields dynamically after task submission.
-

4. THEORY

1. Document Object Model (DOM) Manipulation:

The DOM is a tree-like object representation of an HTML document created by the browser. JavaScript interacts with this tree dynamically:

- `document.getElementById(id)` selects a specific unique element.
- `document.createElement(tagName)` instantiates new element nodes in memory (such as `<li>`, `<span>`, `<button>`).
- `parent.appendChild(child)` attaches newly created elements into the active DOM hierarchy.
- `node.remove()` deletes the selected element node directly from the live DOM tree.

2. Event Handling:

The `addEventListener()` method registers event listener functions to HTML elements. When a user clicks a button, the browser triggers the associated callback function to execute task creation, completion toggling, or deletion.

3. Class Toggling & State Management:

`classList.toggle("completed")` adds or removes the CSS class dynamically, applying or removing the text-decoration: line-through styling without hardcoded inline CSS.

User Input → Add Button Click → Input Validation → Create DOM Nodes → Attach Event Listeners → Append to Task List → Update Empty State

5. ALGORITHM / PROCEDURE

1. Create a project folder containing `index.html`, `style.css`, and `script.js`.
2. **HTML Structure (`index.html`):**
 - Create a heading `<h1>My To-Do List</h1>`.
 - Add an `<input type="text" id="taskInput">` and `<button id="addButton">`.
 - Add a placeholder message `<p id="emptyMessage">No tasks available.</p>`.
 - Define an unordered list container `<ul id="taskList"></ul>`.
3. **CSS Styling (`style.css`):**
 - Center-align content with Arial font family.
 - Style the input field, buttons, and list items with padding and margins.
 - Define `.completed` with text-decoration: line-through and gray color.
4. **JavaScript Logic (`script.js`):**
 - Retrieve DOM references for `taskInput`, `addButton`, `taskList`, and `emptyMessage`.
 - Attach a click event listener to `addButton`.
 - Sanitize input using `.trim()`; if empty, display an alert popup and return.
 - Create `<li>`, `<span>`, `completeButton`, and `deleteButton`.
 - Add a click listener to `completeButton` that executes `span.classList.toggle("completed")`.

- Add a click listener to deleteButton that executes li.remove() and checks taskList.children.length === 0 to restore emptyMessage.
 - Append children into li, append li to taskList, hide emptyMessage, and clear taskInput.value.
5. Launch index.html in a web browser and verify task addition, completion, deletion, and validation.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>

<html>

<head>

  <title>My To-Do List</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <h1>My To-Do List</h1>

  <input type="text" id="taskInput" placeholder="Enter a task">

  <button id="addButton">Add Task</button>

  <p id="emptyMessage">No tasks available.</p>

  <ul id="taskList"></ul>

  <script src="script.js"></script>

</body>

</html>
```

style.css

```
body {

  font-family: Arial, sans-serif;
```

```
    text-align: center;
    margin-top: 50px;
}
```

```
input {
    padding: 10px;
    width: 250px;
}
```

```
button {
    padding: 10px;
    margin: 5px;
    cursor: pointer;
}
```

```
li {
    list-style: none;
    margin: 10px;
    padding: 10px;
}
```

```
.completed {
    text-decoration: line-through;
    color: gray;
}
```

script.js

```
const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");
```

```
addButton.addEventListener("click", function () {

    const taskText = taskInput.value.trim();

    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create new list item
    const li = document.createElement("li");

    // Create task text
    const span = document.createElement("span");
    span.textContent = taskText;

    // Create Complete button
    const completeButton = document.createElement("button");
    completeButton.textContent = "Complete";

    // Create Delete button
    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";

    // Complete button event
    completeButton.addEventListener("click", function () {
        span.classList.toggle("completed");
    });

    // Delete button event
    deleteButton.addEventListener("click", function () {
```

```

        li.remove();

        if (taskList.children.length === 0) {
            emptyMessage.style.display = "block";
        }
    });

    // Add elements to list item
    li.appendChild(span);
    li.appendChild(completeButton);
    li.appendChild(deleteButton);

    // Add list item to task list
    taskList.appendChild(li);

    // Hide empty message
    emptyMessage.style.display = "none";

    // Clear input box
    taskInput.value = "";
});

```

7. CODE EXPLANATION

Code / DOM Method	Explanation
<code>document.getElementById(id)</code>	Selects target elements (taskInput, addButton, taskList, emptyMessage) by unique ID.
<code>addButton.addEventListener("click", ...)</code>	Listens for user click events on the Add Task button to trigger the creation workflow.
<code>taskInput.value.trim()</code>	Extracts the text entered by the user and strips leading/trailing whitespaces.

Code / DOM Method	Explanation
<code>document.createElement("li span button")</code>	Dynamically generates new DOM element nodes in browser memory.
<code>span.textContent = taskText</code>	Sets the inner readable plain-text value of the task item securely.
<code>span.classList.toggle("completed")</code>	Adds the completed CSS class if absent, or removes it if present (enabling strike-through toggle).
<code>li.remove()</code>	Directly deletes the specific <code></code> item node from the parent task list memory.
<code>taskList.children.length === 0</code>	Checks if all task items have been removed to re-display the emptyMessage.
<code>taskList.appendChild(li)</code>	Mounts the fully constructed <code></code> node and its action buttons into DOM container.
<code>taskInput.value = ""</code>	Clears the input field after successful task addition for convenient consecutive entries.

8. TEST CASES AND EXECUTION DATA

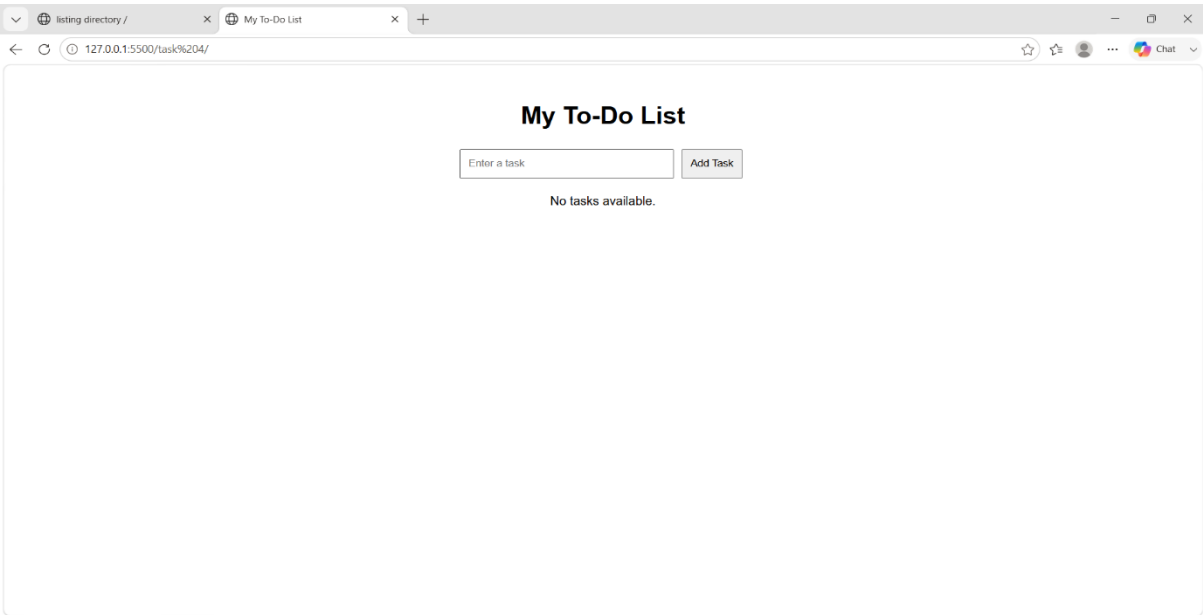
I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Module / Feature	Input / Action	Expected Result	Actual Result
TC-01	Empty Input Validation	Click "Add Task" with blank input	Alert: "Please enter a task."	Alert triggered; execution halted
TC-02	Task Creation	Enter "Study Full Stack", click Add	Appends new task item; hides empty message	Task item created; message hidden
TC-03	Multiple Tasks	Add "Complete Assignment", "Review Lab"	Appends 3 distinct task rows with buttons	All tasks rendered in sequence
TC-04	Complete Task Toggle	Click "Complete" on Task 1	Text becomes gray with strike-through line	.completed class toggled

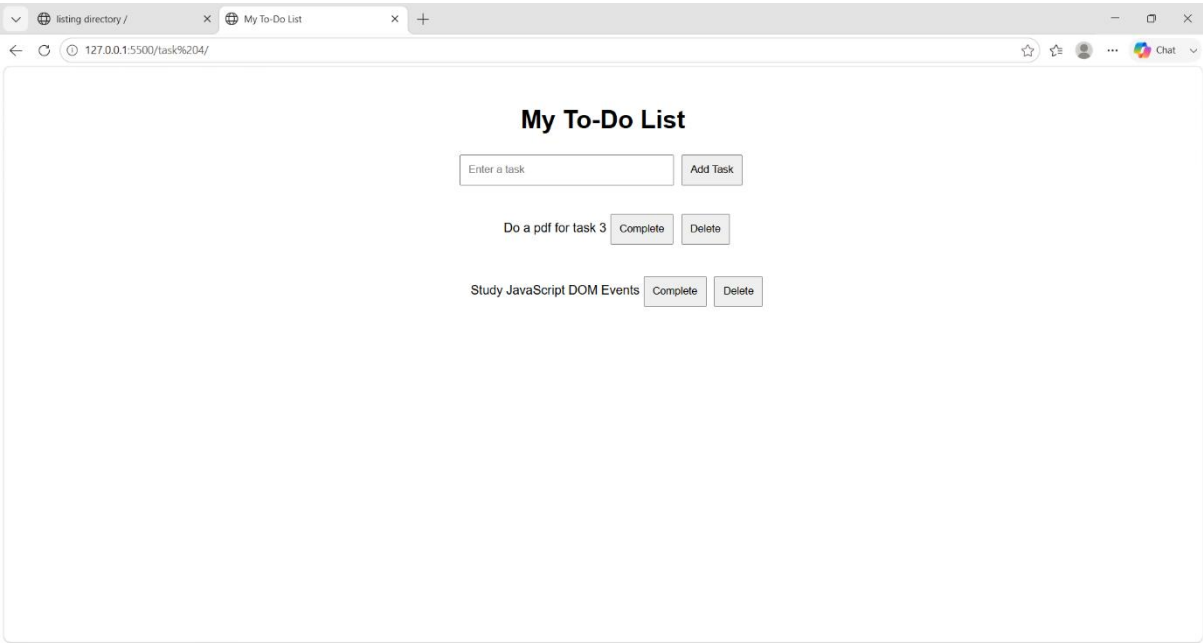
Test Case	Module / Feature	Input / Action	Expected Result	Actual Result
TC-05	Delete Task	Click "Delete" on Task 2	Target task removed from list	Node removed from DOM
TC-06	Empty State Restoration	Delete all remaining tasks	"No tasks available." message reappears	emptyMessage style displayed

9. OUTPUT

Output 1: Initial Empty State & Validation



Output 2: Tasks Added, Completed & Active Items



10. RESULT

Thus, the interactive **To-Do List Web Application** was successfully designed and developed using HTML5, CSS3, and JavaScript DOM manipulation. Features including dynamic task insertion, completion toggling with line-through styling, task deletion, and empty state management were implemented and thoroughly tested across all test cases.
